

Dhirubhai Ambani University

(Formerly known as DA-IICT)

Course: Programming Lab

Dr. Ankit Vijayvargiya

Assistant Professor

Room No. 4205, Faculty Block 4

Email: [ankit_Vijayvargiya\[at\]dau.ac.in](mailto:ankit_Vijayvargiya@dau.ac.in)

Phone: 079-68261628(O), 7877709590(M)

Programming Lab

Course Code : PC503
Course Credit : 1-0-4-3 (L-T-P-Cr)
Course Type : Core

Important Note:

- Keeping more than 80% attendance is required.

Programming Lab

Grading Scheme: Project and Assignments, a mid-semester and a final examination.

Evaluations:

- Hands on Lab and Attendance : 20%
- Project and Assignments : 50%
- End-sem Lab Exam : 30%

Programming Lab

Key Topics:

- Basics of Python Programming
- Core Data Structures in Python
- Object-Oriented Concepts
- Data Analysis for Data Science
- Python Libraries: Pandas, NumPy, Matplotlib, etc.
- Machine Learning Model Implementation
- Deep Learning Model Implementation
- Web Scraping
- Graphical User Interface (GUI) Development

Note: The extent of topic coverage will be influenced by the class's collective engagement and participation.

Programming Lab

Some Project Topics (Not Limited):

- Hand Gesture Recognition
- Emotion Recognition
- Signal Denoising
- Embedded AI and ML
- Voice Recognition
- Image Processing
- Gait Analysis
- Kidney Disease Diagnosis
- Face Recognition
- Biomedical Signal Processing
- Robotics
- Real time weather forecasting

Programming Lab

Books: A single textbook is insufficient for this course. Some of the relevant resources that will be referred to are:

1. Introducing Python: Modern Computing in Simple Packages by Bill Lubanovic, O'Reilly, 2nd Edition, 2019.
2. Coding Python and SQL: How to Install and Python Code: How to Improve SQL and Python by Patricia Jasmine, 1st Edition, 2022.
3. Learning the Pandas Library: Python Tools for Data Munging, Analysis, and Visualization by Matt Harrison and Michael Prentiss, 2nd Edition 2016.
4. Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Ipython by Wes McKinney, Second Edition, 2017.
5. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems by Aurelian Geron, 2nd Edition, 2019.
6. Flask Web Development: Developing Web Applications with Python by Miguel Grinberg, 2nd Edition, 2018.
7. Django3 Web Development Cookbook: Actionable solutions to common problems in Python web development by Aidas Bendoraitis and Jake Kronika, 4th Edition, 2020.
8. Web Scraping with Python: Collecting More Data from the Modern Web by Ryan Mitchell, 1st Edition, 2018.
9. Python GUI Programming - A Complete Reference Guide: Develop responsive and powerful GUI applications with PyQt and Tkinter by Alan D. Moore and B.M. Harwani, 1st Edition, 2019.
10. Python for MATLAB Development by Albert Danial, 1st Edition, Apress Berkeley, CA, 2022.
11. Arduino Cookbook: Recipes to Begin, Expand, and Enhance Your Projects by Michael Margolis, 2nd Edition, 2021.
12. Exploring Arduino: Tools and Techniques for Engineering Wizardry by Jeremy Blum, 2nd Edition, 2019..
13. The Elements of MATLAB Style by Richard K. Johnson, 1st Edition, Cambridge University Press, 2010.

Programming Lab

I am available/approachable at

Room No 4205

Faculty Block-4

Email: `ankit_Vijayvargiya[at]dcu.ac.in`

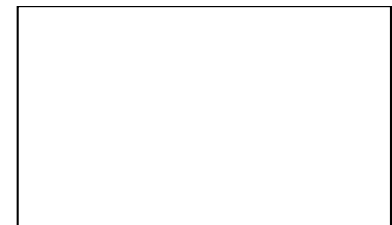
Programming Lab

In Lab, following software platform will be used:

- Python Programming: Jupyter Notebook/ VS code

Google Classroom Code:

ncq2do7r



Thank You

Dhirubhai Ambani University

(Formerly known as DA-IICT)

Topic: Introduction to Python Programming

Dr. Ankit Vijayvargiya

Room No. 4205, Faculty Block 4
Email: `ankit_Vijayvargiya[at]dau.ac.in`

Brief Introduction

❖ Python is a programming language created in 1991 by Guido van Rossum.

Why Python?

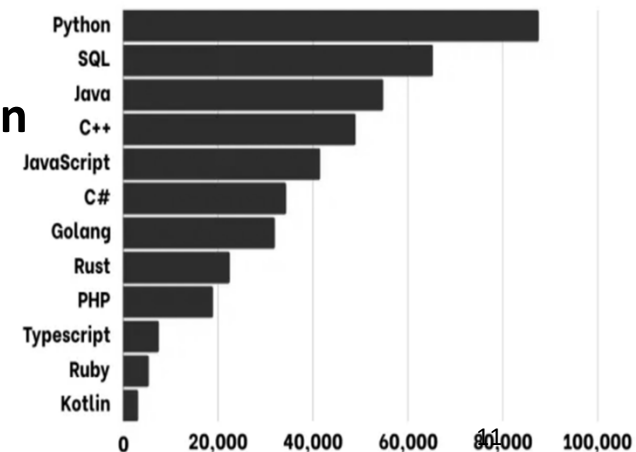
❖ Python is an open-source language.

❖ It works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc.)

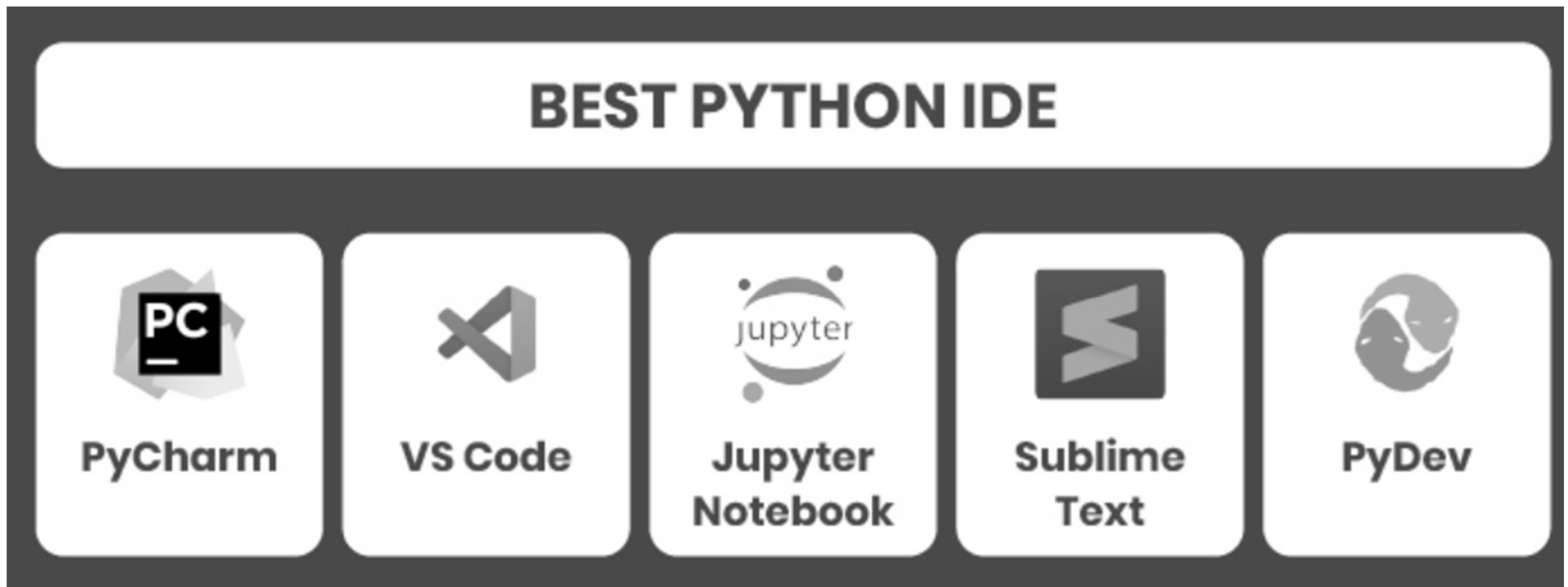
❖ Easier to learn and use.

❖ Powerful libraries – Ready-made ML, AI, web, and automation tools.

❖ High industry demand – More jobs and career opportunities.



Python IDE



Installation of Jupyter Notebook

Download Now

For installation assistance, refer to [Troubleshooting](#).

Download Anaconda Distribution or Miniconda by choosing the proper installer for your machine. Learn the difference from our [Documentation](#).



Anaconda Installers

Download



Windows

Python 3.12

64-Bit Graphical Installer (912.3M)



Mac

Python 3.12

64-Bit (Apple silicon) Graphical Installer (704.7M)

64-Bit (Apple silicon) Command Line Installer (707.3M)

64-Bit (Intel chip) Graphical Installer (734.7M)

64-Bit (Intel chip) Command Line Installer (731.2M)



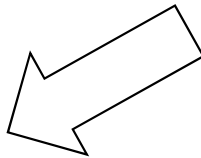
Linux

Python 3.12

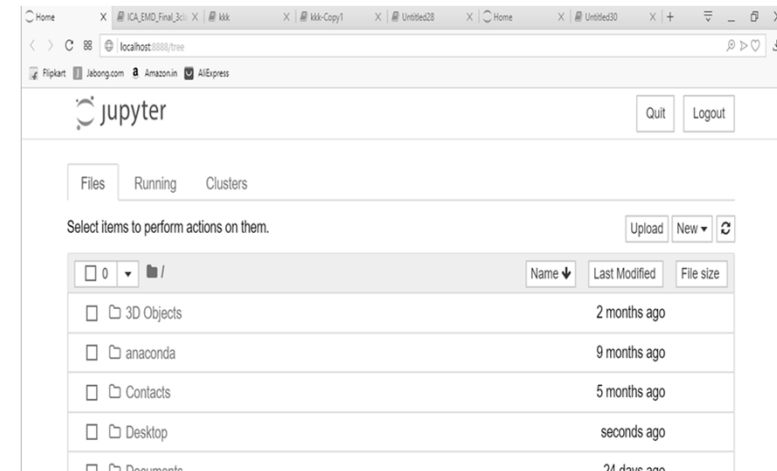
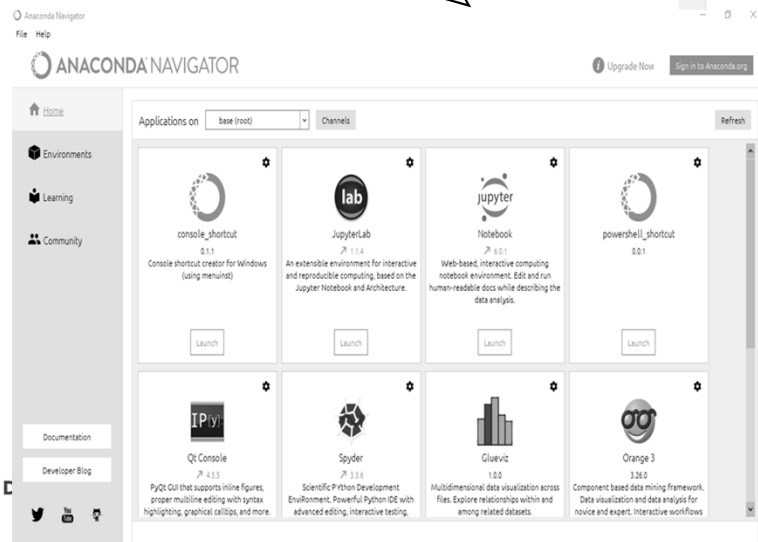
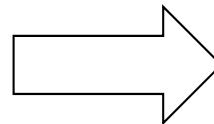
64-Bit (x86) Installer (1007.9M)

64-Bit (AWS Graviton2 / ARM64) Installer (800.6M)

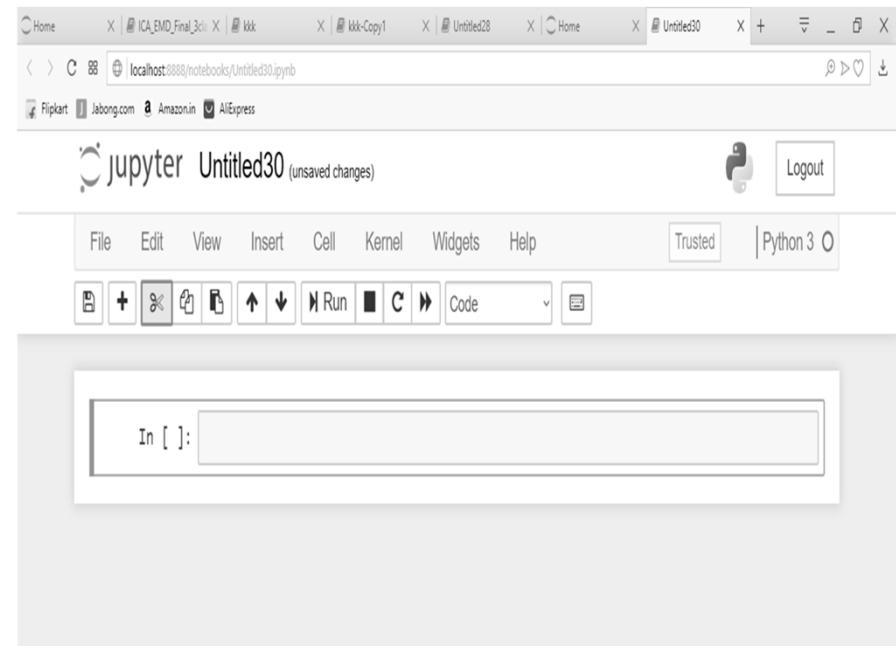
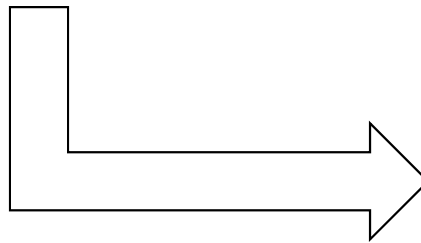
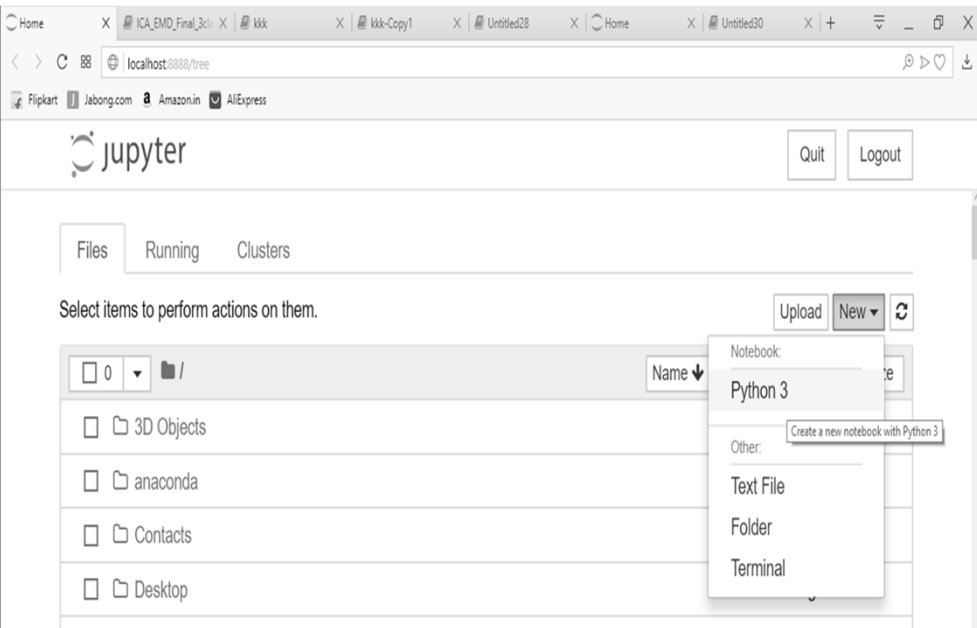
64-bit (Linux on IBM Z & LinuxONE) Installer (425.8M)



Source: anaconda.com



Create a new program file in Jupyter Notebook



Python Vs Other Languages

❖ On average, Python code is smaller than JAVA/C++ codes by 3.5 times.

C Code

```
#include <stdio.h>
void main()
{
    printf("Hello World");
}
```

C++ code:

```
#include <iostream>
void main( )
{
    cout << "Hello World";
}
```

Python code:

```
print("Hello World")
```

❖ Python is interpreted while C/C++, etc. are compiled.

- **Compiler:** spends a lot of time analysing and processing the program, faster program execution
- **Interpreter:** relatively little time is spent analysing and processing the program, slower program execution

Print in Python

`print('Hello World')`
or
`print("Hello World")`

```
print("Hello World")
```

Hello World

`print(Item1 , Item2)`

```
: a=2  
b=5  
print(a)  
print(b)
```

2
5

```
: a=2  
b=5  
print(a,b)
```

2 5

Print variable with statements:

```
print('The value of x is {} and y is {}'.format(2,5))
```

The value of x is 2 and y is 5

```
: a=2  
b=5  
print('The value of x is {} and y is {}'.format(a,b))
```

The value of x is 2 and y is 5

Comments and Indentation

- ❖ For single line comment hash (#) symbol is used.
- ❖ For multi-line comments use triple quotes, either ''' or """".

```
# WAP to print Hello World  
print("Hello World")
```

```
"""WAP  
to print  
Hello World"""  
print("Hello World")
```

- ❖ Python uses indentation instead of braces to determine the scope of expressions.
- ❖ All lines must be indented the same amount to be part of the scope (or indented more if part of an inner scope).

```
[45]: print("my name is Ram")  
      print("my age is 20")  
      print(c)
```

```
Cell In[45], line 2  
    print("my age is 20")  
    ^
```

```
IndentationError: unexpected indent
```

Exercise- 1

WAP to print the following text in the given format:

Twinkle, twinkle, little star, How I wonder what you are! Up above the world so high,
Like a diamond in the sky. Twinkle, twinkle, little star, How I wonder what you are

```
Twinkle, twinkle, little star,  
How I wonder what you are!  
Up above the world so high,  
Like a diamond in the sky.  
Twinkle, twinkle, little star,  
How I wonder what you are
```

```
print("Twinkle, twinkle, little star, \nHow I wonder what you are! \nUp above the world  
so high, \nLike a diamond in the sky. \nTwinkle, twinkle, little star, \nHow I wonder  
what you are")
```

Exercise- 2

WAP to print the following text in the given format:

Twinkle, twinkle, little star, How I wonder what you are! Up above the world so high,
Like a diamond in the sky. Twinkle, twinkle, little star, How I wonder what you are

```
Twinkle, twinkle, little star,  
    How I wonder what you are!  
        Up above the world so high,  
            Like a diamond in the sky.  
Twinkle, twinkle, little star,  
    How I wonder what you are
```

```
print("Twinkle, twinkle, little star, \n\tHow I wonder what you are! \n\t\tUp above the  
world so high, \n\t\t\tLike a diamond in the sky. \nTwinkle, twinkle, little star, \n\tHow  
I wonder what you are")
```

Variables

- ❖ A variable name can only contain alpha-numeric characters and underscores.
- ❖ A variable name cannot start with a number.
- ❖ There is no need to declare the type of a variable during initialization.
- ❖ Variable names are case-sensitive (age, Age, and AGE are three different variables)
- ❖ Reserved words such as break, for, continue, etc., can't be used as variable names.
- ❖ In Python, variable types can change during runtime (dynamic typing).
- ❖ In Python, multiple assignments can be made in a single statement.

```
Age = 4 # x is of type int  
x = "SHYAM" # x is now type str
```

```
print (Age)      print(x)
```

```
Output: 4        SHYAM
```

```
x = 4 # x is of type int  
x = "SHYAM" # x is now type str
```

```
print (x)
```

```
Output: SHYAM
```

```
a, b, c = 5, 3.2, "Hello"
```

```
Print(a, b ,c)
```

```
Output: 5 3.2 Hello
```

Input from user

- ❖ In Python, we use the `input( )` function to take input from the user.
- ❖ Whatever you enter as input, the input function converts it into a string.
- ❖ If you enter an integer value, the `input()` function converts it into a string.

```
print("what is your name")
input()
```

what is your name

↑↓ for history. Search history w:

```
: print("what is your name")
input()
```

what is your name

Radheshyam

```
: 'Radheshyam'
```

- ❖ Once the value comes into the variable string, it can be converted into an integer or float, etc.

```
: print("what is your Age")
Age=input()
x=int(Age)
print(x)
```

what is your Age

25

25

```
print("what is your Salary")
Salary=input()
x=float(Salary)
print(x)
```

what is your Salary

20000

20000.0

Input from user

- ❖ We can use the `int()` function before `input()` function to accept an integer from the keyboard as:

```
print("what is your Age")
Age=int(input())
print(Age)
```

```
what is your Age
25
25
```

- ❖ Similarly, we can use the `float()` function before `input()` function to accept a float value from the keyboard as:

```
: print("what is your Salary")
Salary=float(input())
print(Salary)
```

```
what is your Salary
20000
20000.0
```

Exercise- 3

Write a program that accepts the radius of a circle from the user and computes the area.

```
radius_str = input("Enter the radius of the circle: ")
radius = float(radius_str)
area = (22/7) * radius ** 2
print(area)
```

```
Enter the radius of the circle:  1.11
3.8723142857142863
```

Exercise- 4

Write a Python program that accepts the user's first and last name and prints them in reverse order.

Dhirubhai Ambani University

(Formerly known as DA-IICT)

Topic: Operators

Dr. Ankit Vijayvargiya

Assistant Professor

Room No. 4205, Faculty Block 4

Email: [ankit_Vijayvargiya\[at\]dau.ac.in](mailto:ankit_Vijayvargiya[at]dau.ac.in)

Operators

- An operator is a symbol that represents an operation that may be performed on one or more operands.
- An operand is a value that a given operator is applied to.
- Example: $4+3*k$, + and * are the operators and 4, 3, and k are the operands
- Types of Operator
 - ❖ Arithmetic Operators
 - ❖ Comparison Operators
 - ❖ Logical Operators
 - ❖ Assignment Operators
 - ❖ Bitwise operators
 - ❖ Membership Operators
 - ❖ Identity Operators

Arithmetic Operators

Arithmetic operators are used with numeric values to perform common mathematical operations:

Operator	Name	Example
+	Addition	$x + y$
-	Subtraction	$x - y$
*	Multiplication	$x * y$
/	Division	x / y
%	Modulus	$x \% y$
**	Exponentiation	$x ** y$
//	Floor division	$x // y$

Arithmetic Operators

Program to Accept Two Integer Values from the user and Perform Arithmetic Operations

```
a=int(input("Input a"))  
b=int(input("Input b"))  
c=a+b  
print(c)
```

Input a-2
Input b2
0

```
a=int(input("Input a"))  
b=int(input("Input b"))  
c=a-b  
print(c)
```

Input a7
Input b5
2

```
a=int(input("Input a"))  
b=int(input("Input b"))  
c=a*b  
print(c)
```

Input a2
Input b3
6

```
a=int(input("Input a"))  
b=int(input("Input b"))  
c=a/b  
print(c)
```

Input a5
Input b2
2.5

```
a=int(input("Input a"))  
b=int(input("Input b"))  
c=a%b  
print(c)
```

Input a5
Input b2
1

```
a=int(input("Input a"))  
b=int(input("Input b"))  
c=a//b  
print(c)
```

Input a5
Input b2
2

Comparison/Relational Operators

Comparison operators are used to compare two values:

Operator	Name	Example
==	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Comparison/Relational Operators

Program to Accept Two Integer Values from the user and Perform Comparison Operations

```
a=int(input("Input a"))
b=int(input("Input b"))
if a==b:
    print(a+b)
else:
    print(a-b)
```

Input a5
Input b3
2

```
a=int(input("Input a"))
b=int(input("Input b"))
if a<b:
    print(a+b)
else:
    print(a-b)
```

Input a3
Input b2
1

```
a=int(input("Input a"))
b=int(input("Input b"))
if a!=b:
    print(a+b)
else:
    print(a-b)
```

Input a5
Input b3
8

```
a=int(input("Input a"))
b=int(input("Input b"))
if a>=b:
    print(a+b)
else:
    print(a-b)
```

Input a3
Input b3
6

```
a=int(input("Input a"))
b=int(input("Input b"))
if a>b:
    print(a+b)
else:
    print(a-b)
```

Input a3
Input b2
5

```
a=int(input("Input a"))
b=int(input("Input b"))
if a<=b:
    print(a+b)
else:
    print(a-b)
```

Input a3
Input b2
1

Logical Operators

Logical operators are used to combine conditional statements:

Operator	Description	Example
and	Returns True if both statements are true	$x < 5$ and $x < 10$
or	Returns True if one of the statements is true	$x < 5$ or $x < 4$
not	Reverse the result, returns False if the result is true	not($x < 5$ and $x < 10$)

Logical Operators

Program to Accept Two Integer Values from the user and Perform Logical Operations

```
a=int(input("Input a"))
b=int(input("Input b"))
if a>5 and b<5:
    print(a+b)
else:
    print(a-b)
```

Input a9
Input b6
3

```
a=int(input("Input a"))
b=int(input("Input b"))
if a>5 or b<5:
    print(a+b)
else:
    print(a-b)
```

Input a9
Input b6
15

```
a=int(input("Input a"))
b=int(input("Input b"))
if not(a>5 or b<5):
    print(a+b)
else:
    print(a-b)
```

Input a9
Input b6
3

Assignment Operators

❖ Assignment operators is used to assign values to the variables.

Operator	Example	Equals To
=	a = 10	a = 10
+=	a += 10	a = a+10
-=	a -= 10	a = a-10
*=	a *= 10	a = a*10
/=	a /= 10	a = a / 10
%=	a %= 10	a = a % 10
//=	a //= 10	a = a // 10
**=	a **= 10	a = a ** 10
&=	a &= 10	a = a & 10
=	a = 10	a = a 10
^=	a ^= 10	a = a ^10
>>=	a >>= 10	a = a >> 10
<<=	a <<= 10	a = a << 10

Bitwise Operators

- ❖ Bitwise operators act on operands as if they are a string of binary digits.
- ❖ It operates bit by bit.

Operator	Example	Equals To
=	a = 10	a = 10
+=	a += 10	a = a+10
-=	a -= 10	a = a-10
*=	a *= 10	a = a*10
/=	a /= 10	a = a / 10
%=	a %= 10	a = a % 10
//=	a //= 10	a = a // 10
**=	a **= 10	a = a ** 10
&=	a &= 10	a = a & 10
=	a = 10	a = a 10
^=	a ^= 10	a = a ^10
>>=	a >>= 10	a = a >> 10
<<=	a <<= 10	a = a << 10

>>

Bitwise right shift

<<

Bitwise left shift

Bitwise Operators

Program to Accept Two Integer Values from the user and Perform Bitwise Operations

```
a = int(input("Input a: "))
b = int(input("Input b: "))

print("Bitwise AND (a & b):", a & b)
print("Bitwise OR (a | b):", a | b)
print("Bitwise XOR (a ^ b):", a ^ b)
print("Bitwise NOT (~a):", ~a)
print("Bitwise Left Shift (a << 1):", a << 1)
print("Bitwise Right Shift (a >> 1):", a >> 1)
```

```
Input a: 5
Input b: 2
Bitwise AND (a & b): 0
Bitwise OR (a | b): 7
Bitwise XOR (a ^ b): 7
Bitwise NOT (~a): -6
Bitwise Left Shift (a << 1): 10
Bitwise Right Shift (a >> 1): 2
```

In binary

5= 0101

2= 0010

AND: 0000 =0

OR: 0111=7

XOR: 0111=7

NOT: 1010= -6 (2's
Complement)

Left: 1010=10

Right: 0010=2

Membership Operators

- ❖ `in` and `not in` are the membership operators in Python. They are used to test whether a value or variable is found in a sequence (string, list, tuple, set and dictionary).

```
In [20]: x = 'Hello world'
         print('h' in x)
```

False

```
In [19]: print('H' in x)
```

True

```
In [21]: print('K' not in x)
```

True

Exercise- 3

Write a program that checks if a given character exists in a string. Take input from the user for the string and the character.

```
: text=input("Enter the text")
  char= input("Enter a character to check")
  print(char in text)
```

Enter the text hi shyam

Enter a character to check a

True

Identity Operators

- ❖ `is` and `is not` are the identity operators in Python, not the equality operator. They check whether **two variables refer to the exact same object in memory**, not just if their contents are equal.

```
In [22]: x = ["apple", "banana"]
         y = ["apple", "banana"]
         z = x
         print(x is z)
```

True

```
In [23]: print(x is y)
```

False

```
x=["apple", "banana"]
y=["apple", "banana"]
z=x
print(id(x)) # Memory address of x
print(id(y)) # Memory address of y
print(id(z)) # Memory address of z
```

2063128871104

2063182771264

2063128871104